



Clash Royale AI



Dr. Yilmaz

Ezra Gilchrest
Computer Systems Lab

TJSTAR 2026

Period: 7

Background

Competitive Battle Royale



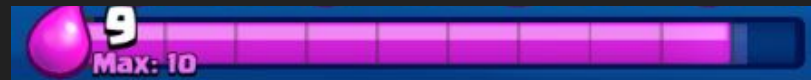
VS



Build a deck of 8 from 121 unique cards



Elixir Bar



Background



Background



Rolls a rock that
hits many troops



Many units with
low health

Famous (Meta) Decks



Deck Used by the AI



VS

Deck Used by AI's Opponent



Problem: No existing autonomous, open source Clash Royale bot



Goal: Using reinforcement learning, create a competitive Clash Royale AI that mimics human play and can win at least one of ten games against a human.

Challenges

- Must use android emulator (Bluestacks)
- No official Supercell API
- Enormous decision space
 - 121 unique cards
 - 1000+ tiles
- Training limited by real-time gameplay speed (~3-4 min/game through BlueStacks)



Other Solutions

AI in Other Competitive Games

- DeepMind's AlphaStar (2019) defeated professional StarCraft II players using reinforcement learning + computer vision
- OpenAI Five (2018) beat human Dota 2 teams using proximal policy optimization (PPO)
- Key challenge across all game AI: real-time decision making under incomplete information
- Clash Royale shares these challenges: partial observability, time pressure, complex card interactions



Basic Clash Royale Bot

Pros:

- Easy to use
- Safe environment

Cons:

- Deterministic behavior makes it trivially exploitable. No game state modeling
- Cannot adapt to opponent deck. This violates the core strategic requirement of CR
- Absence of open-source code prevents reproducibility or extension by researchers



Ombarus

Pros:

- Able to understand troops and elixir
- Able to play cards on screen
- Technically able to play multiple decks

Cons:

- Hard coded actions grants no merit to play against
- Lacks any threat detection or elixir optimization
- Not reproducible or extendable



Link to full video: <https://www.youtube.com/watch?v=Q4cVkJFjvCo>

Junior Builds

Pros:

- Able to understand troops and elixir
- Able to play cards on the screen
- Uses reinforcement learning

Cons:

- Uses Roboflow (third-party hosted service) for detection, no custom trained model
- No centroid tracking, model cannot distinguish ally vs enemy troops
- No tower health detection; cannot compute a meaningful reward signal
- Uses Q-learning and epsilon-greedy policy -> doesn't generalize well



Why our approach is different

- Junior Builds detects troops but cannot tell ally from enemy. The agent cannot model the battlefield
- Junior Builds reward signal is based on win/loss only (no intermediate feedback from tower damage)
- Our agent receives dense rewards from tower health changes every frame, enabling faster learning
- PPO rather than Q learning



**IF YOU CANNOT DETERMINE
THE DIFFERENCE BETWEEN
YOUR ENEMY AND ALLY,
YOUR CLASH ROYALE AI WILL SUCK**

-SUN TZU, THE ART OF WAR

Gap in Existing Work

- All three prior solutions lack a complete game state representation (troop type, position, elixir, hand)
- None are publicly available or reproducible
- No existing bot combines a full CV pipeline with a PPO reinforcement learning agent trained on real gameplay



Novelty

1. Project implements ally/enemy classification via centroid tracking with spawn-side team locking
2. Incorporates tower health detection into the game state, enabling a meaningful RL reward signal
3. Richer structured state representation than any prior work: troop type, position, team, elixir, hand, and tower health



Feature	Basic Bot	Ombars	Junior Builds	My project
Elixir Tracking	No	Yes	Yes	Yes
Ally/enemy classification	No	No	No	Yes
Tower health detection	No	No	No	Yes
Structured game state	No	Partial	Partial	Yes
Reinforcement learning	No	No	Yes	Yes
Publicly reproducible	No	No	No	Yes

Impact/Importance

1. Other competitive bots:
 - a. Chess, Go
 - b. Counter-Strike 2
 - c. Rocket league
2. Real time decision making
3. Game complexity
4. Insight to strategic optimization

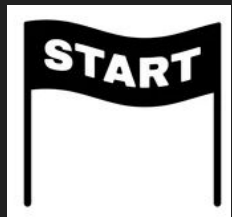


Source: <https://www.regencychess.com/blog/best-chess-player-in-the-world/>



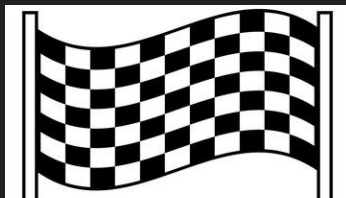
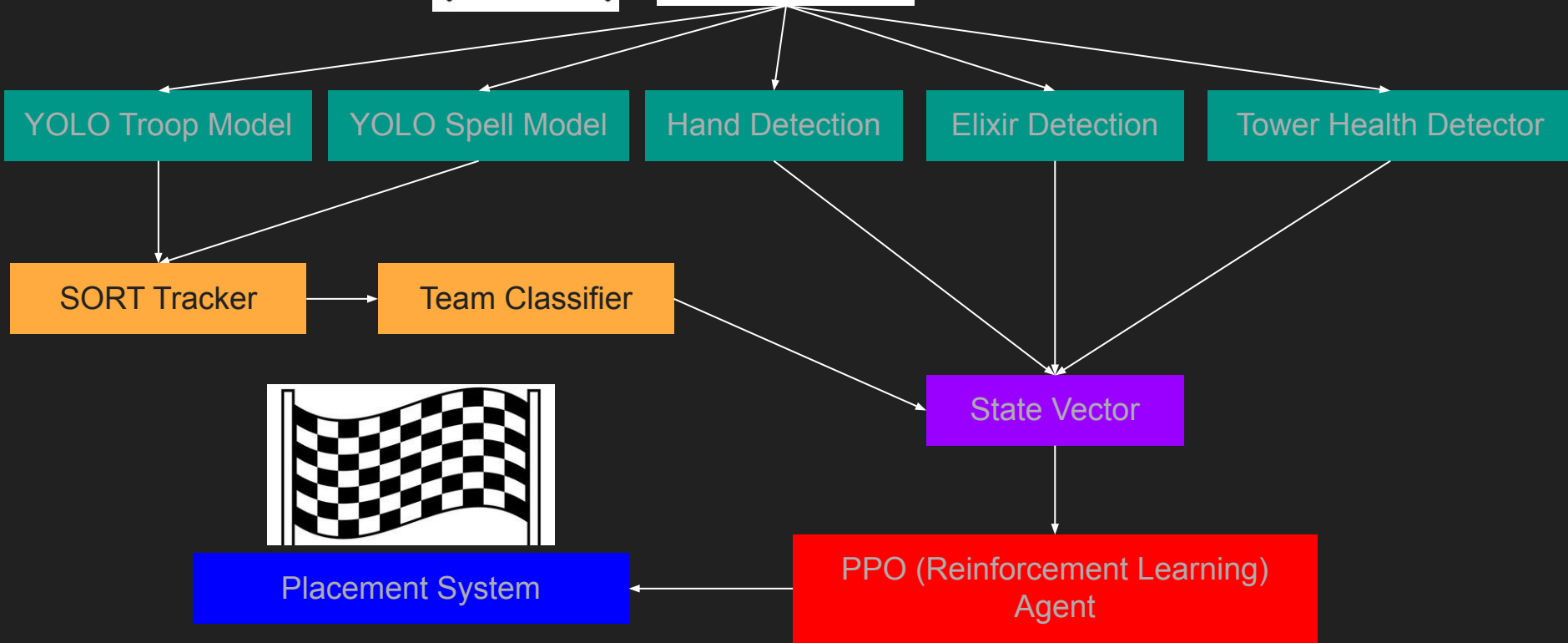
Source: <https://dmcommunity.org/2020/09/15/real-time-decision-making/>

METHODS



Data Flow

Screen Capture



Placement System

PPO (Reinforcement Learning)
Agent

YOLO Model Image Detection

Enemy troops

Team troops

Tower Health/State

Card Cycle

Elixir



YOLO Detection

1. Gather Data

- a. Record game on bluestacks
- b. Cut out parts of video without moving troops/spells (clipchamp)
- c. Parse video into frames
 - i. Troop 1/s (292 Frames)
 - ii. Spell 5/s (576 Frames)

FOR each video:

OPEN video

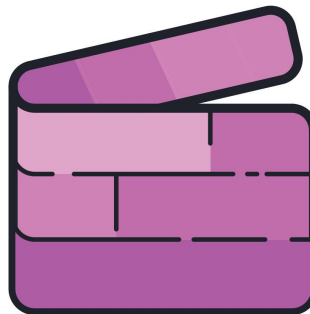
FOR each frame:

IF frame meets correct time interval:

SAVE frame as image

CLOSE video

PRINT total frames saved



YOLO Detection

2. Label Data (label studio)

- Create class labels
- Manually go through each frame creating bounding boxes for each troop

valkyrie 1	giant 2	goblin 3
hog_rider 4	cannon 5	mini_peka 6
mega_minion 7	musketeer 8	knight 9
archer 0	golbin_hut q	spear_goblin w





YOLO Troop Detection

3. Train YOLO model (CNN)

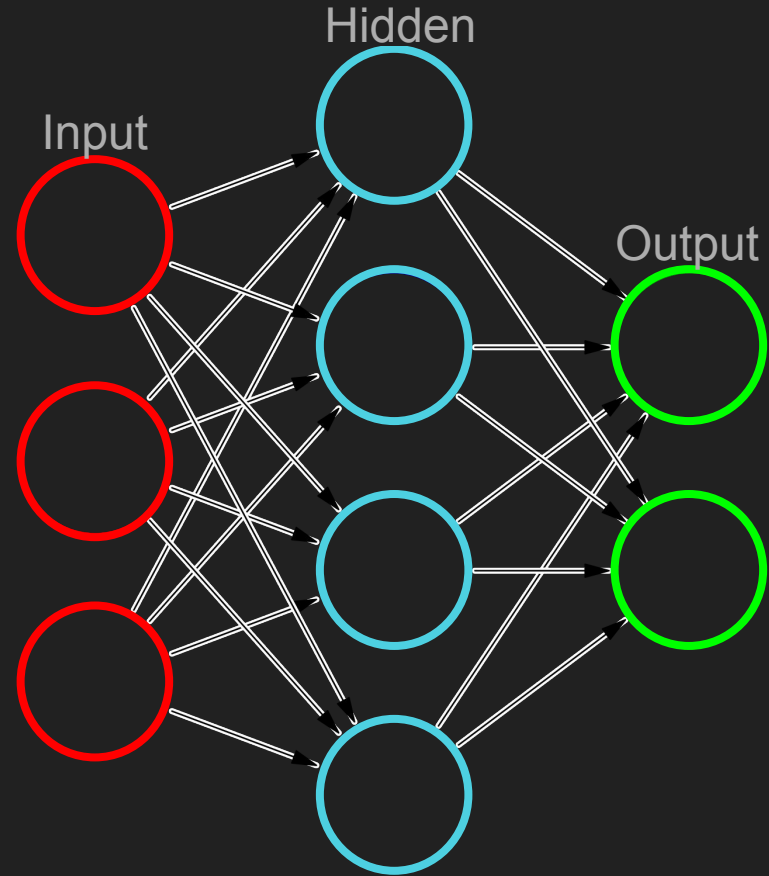
Data split: 80% training 15% validation 5% test

Weight initialization: pre-trained YOLO weights

Epochs: 75

Batch Size: 8 images per batch

Learning rate: 0.001



Elixir Detection

1. Blob image is always in the same location relative to the rest of the map
2. Detect blob to track orientation of overall image using template matching (directly loops through pixels to match one image onto another)
3. Loop through individual pixels, checking for specific color



Hand Detection

1. Cards are detected using template matching
 - a. Only eight cards to search for
 - b. Cards always look the same
 - c. If there is not enough elixir to place a card, it will not be detected
2. Sort X coordinates to determine order

Example, all cards are playable



Only some playable cards



Tower Health Detection

- Tower health bars are always in a fixed pixel location relative to the arena
- Sample a horizontal strip of pixels across the health bar region
- Count how many pixels match the health bar color using HSV color detection (same method as elixir)
- Divide colored pixels by total pixels in the strip to get a 0 to 1 health percentage
- Sampled every time a state vector is built. This gives the RL agent a continuous signal of damage dealt and received



Reinforcement Learning: State, Action, Reward

State vector (260 values, all normalized 0–1):

- [0:210] Up to 30 troops × 7 features each: x position, y position, team (0=ally, 1=enemy), unit type (normalized index over all YOLO classes), and 3 additional feature slots
- [210:246] Cards in hand: 4 cards, each one-hot encoded over 9 possible card types
- [246:255] Next card: one-hot encoded over 9 possible card types
- [255] Current elixir (divided by 10)
- [256:260] Four tower healths: ally left, ally right, enemy left, enemy right (each as % of max)

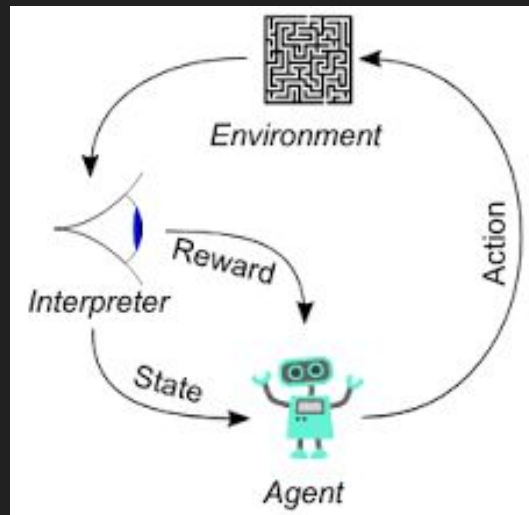
Action space (9 discrete actions):

- 4 card slots × 2 lanes (left, right) = 8 play actions
- 1 do-nothing action
- Actions are masked by elixir cost via MaskablePPO

Reward function:

- Enemy tower damage this frame × 0.01
- Ally tower damage this frame × -0.01
- Win: +10, Loss: -10
- Dense per-frame reward teaches the agent faster than win/loss alone;

tower health delta is computed each frame from `get_tower_health()`



Proximal Policy Optimization (PPO)

What:

- PPO learns through trial and error.
- Makes gradual updates for stable learning.

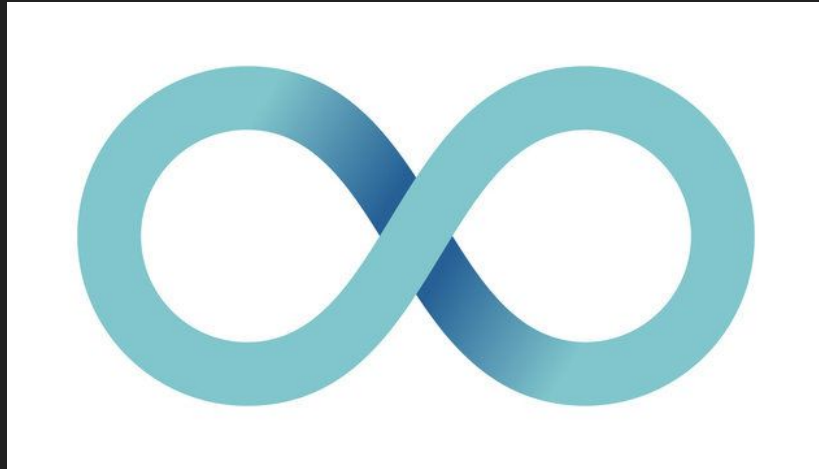
Advantages of PPO:

- Handles complex game states well.
- Supports action masking with MaskablePPO.
- Works well with delayed rewards.
- More scalable and stable than Q-learning for Clash Royale.



PPO Training loop

1. Agent reads current state vector from the vision pipeline
2. PPO policy outputs one of 25 actions
3. Action is executed via pyautogui (card click + arena click)
4. Vision pipeline reads next state and computes reward from tower health delta
5. Experience is stored and model weights update after each game



DEMO

✱ [Return to Table of Contents](#) ✱

© 2000 Blackwell Science Ltd

1997

© 2004 Blackwell Publishing Ltd, *Journal of Internal Medicine* 255: 105–112

Accepted manuscript to appear in JGIM

Enclosed is payment of \$

Author(s)	James M. Bracken
Editor(s)	James M. Bracken

Author(s)	James A. Brinkmann
School(s)	University of Minnesota

Edward J. Cummings, Jr.

Received 10/1/2003
 Accepted 10/1/2003

1000

[illegible]

© 2005 Blackwell Publishing Ltd, *Journal of Internal Medicine* 258: 103–110

Accepted manuscript to appear in JGIM

Received 1 September 2004; accepted 12 November 2004
Published online 12 December 2004 in Wiley InterScience (www.interscience.wiley.com). DOI: 10.1002/anie.200461000

Year	Number of cases	Percentage of cases
1990	10	10.0
1991	15	15.0
1992	20	20.0
1993	25	25.0
1994	30	30.0
1995	35	35.0
1996	40	40.0
1997	45	45.0
1998	50	50.0
1999	55	55.0
2000	60	60.0
2001	65	65.0
2002	70	70.0
2003	75	75.0
2004	80	80.0
2005	85	85.0
2006	90	90.0
2007	95	95.0
2008	100	100.0
2009	105	105.0
2010	110	110.0
2011	115	115.0
2012	120	120.0
2013	125	125.0
2014	130	130.0
2015	135	135.0
2016	140	140.0
2017	145	145.0
2018	150	150.0
2019	155	155.0
2020	160	160.0
2021	165	165.0
2022	170	170.0
2023	175	175.0
2024	180	180.0
2025	185	185.0
2026	190	190.0
2027	195	195.0
2028	200	200.0
2029	205	205.0
2030	210	210.0
2031	215	215.0
2032	220	220.0
2033	225	225.0
2034	230	230.0
2035	235	235.0
2036	240	240.0
2037	245	245.0
2038	250	250.0
2039	255	255.0
2040	260	260.0
2041	265	265.0
2042	270	270.0
2043	275	275.0
2044	280	280.0
2045	285	285.0
2046	290	290.0
2047	295	295.0
2048	300	300.0
2049	305	305.0
2050	310	310.0
2051	315	315.0
2052	320	320.0
2053	325	325.0
2054	330	330.0
2055	335	335.0
2056	340	340.0
2057	345	345.0
2058	350	350.0
2059	355	355.0
2060	360	360.0
2061	365	365.0
2062	370	370.0
2063	375	375.0
2064	380	380.0
2065	385	385.0
2066	390	390.0
2067	395	395.0
2068	400	400.0
2069	405	405.0
2070	410	410.0
2071	415	415.0
2072	420	420.0
2073	425	425.0
2074	430	430.0
2075	435	435.0
2076	440	440.0
2077	445	445.0
2078	450	450.0
2079	455	455.0
2080	460	460.0
2081	465	465.0
2082	470	470.0
2083	475	475.0
2084	480	480.0
2085	485	485.0
2086	490	490.0
2087	495	495.0
2088	500	500.0
2089	505	505.0
2090	510	510.0
2091	515	515.0
2092	520	520.0
2093	525	525.0
2094	530	530.0
2095	535	535.0
2096	540	540.0
2097	545	545.0
2098	550	550.0
2099	555	555.0
2100		

Received 12 November 2003; accepted 10 February 2004

Received 10 September 2001; accepted 10 October 2001

Adjusted R-squared	0.40	0.40
F-statistic	10.00	10.00

Table 1

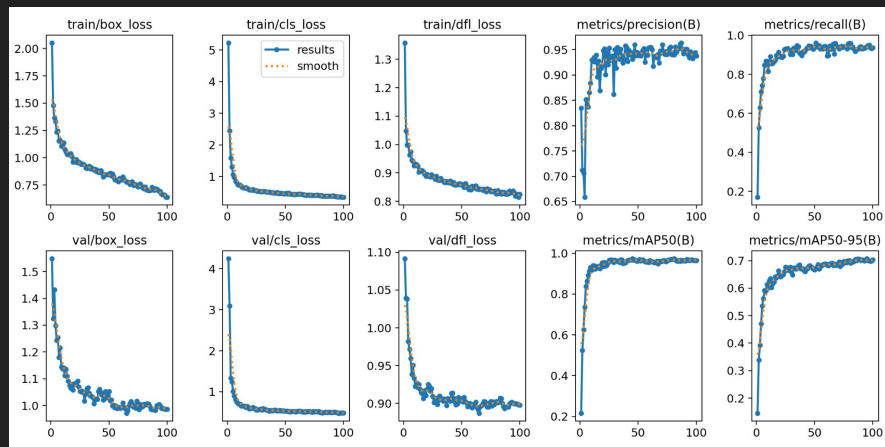
目次



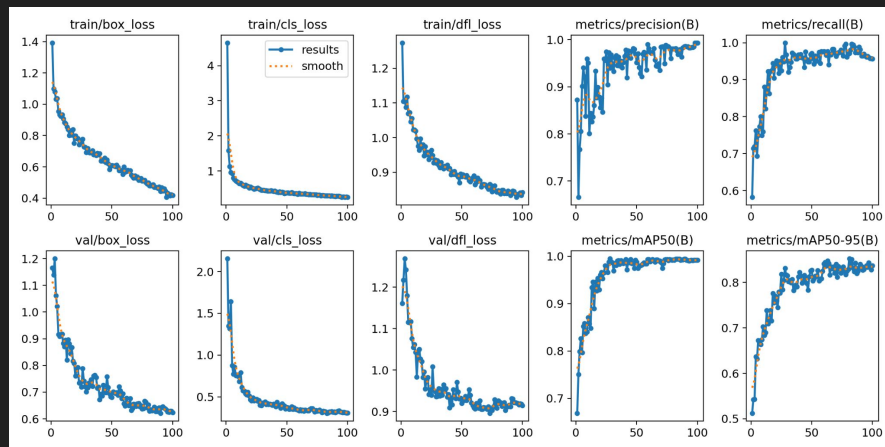
RESULTS

Model	Accuracy
YOLO troop detection (tested on 10 images)	85% (Troop) 92% (Spell)
Elixir detection	100%
Hand detection	100%
Tower Health	91%

Troops



Spells

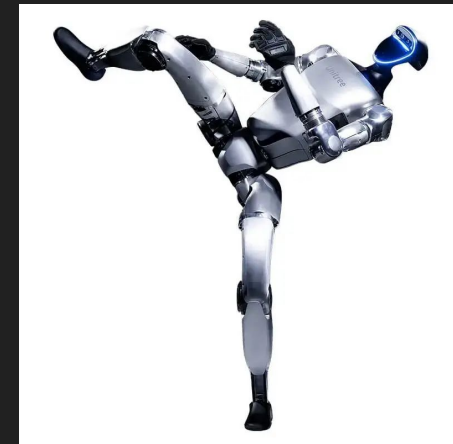


RESULTS

- Correctly identifies what is on screen
- Appropriately assigns rewards
- Has not developed a working strategy to successfully win in games

Basic Clash Royale bot win rate against human with 13.5k trophies (out of 10 games): 0%

My AI win rate against human with 13.5k trophies (out of 10 games): 10%



WHY IS PERFORMANCE SO LOW?

- State vector is still extremely large and difficult for a PPO to understand within a reasonable amount of training time
- Clash Royale requires specific strategies and understand of the relationships between cards, something that an AI will struggle to learn.
- Overall, 10% is impressive!



Limitations

- Model trained on one fixed deck against one fixed opponent deck (limited strategy and generalization to other decks)
- YOLO model does not include different versions of cards (heroes, evolutions)

Future Work

1. Enable the AI to recognize all 121 troops and spells
2. Develop the AI to adapt strategies against different decks
3. Allow the AI to play various meta decks without retraining



Source: <https://www.istockphoto.com/photos/exciting-homework>

REFERENCES

- TensorFlow. (n.d.). Introduction to reinforcement learning with TensorFlow agents. Retrieved April 6, 2026, from https://www.tensorflow.org/agents/tutorials/0_intro_rl
- Baeldung. (n.d.). Q-learning vs. Deep Q-learning vs. Deep Q-network. Retrieved April 6, 2026, from <https://www.baeldung.com/cs/q-learning-vs-deep-q-learning-vs-deep-q-network>
- Wikipedia contributors. (n.d.). Monte Carlo tree search. In Wikipedia. Retrieved March 25, 2026, from https://en.wikipedia.org/wiki/Monte_Carlo_tree_search
- AI Mind. (n.d.). Combining MCTS and reinforcement learning. Retrieved March 25, 2026, from <https://pub.aimind.so/combining-mcts-and-reinforcement-learning-09aac8c46b58>
- National Center for Biotechnology Information. (n.d.). Reinforcement learning and MCTS research article. PubMed. Retrieved March 25, 2026, from <https://pubmed.ncbi.nlm.nih.gov/38770719/>
- Massachusetts Institute of Technology Professional Education. (n.d.). Advanced reinforcement learning. Retrieved March 23, 2026, from <https://professional.mit.edu/course-catalog/advanced-reinforcement-learning>
- Ursina Sanderink. (n.d.). Reinforcement learning methods, applications, and modern techniques. Medium. Retrieved March 23, 2026, from <https://medium.com/@sanderink.ursina/reinforcement-learning-methods-applications-and-modern-techniques-d2b4cc51eefe>
- Wikipedia contributors. (n.d.). Proximal policy optimization. In Wikipedia. Retrieved March 16, 2026, from https://en.wikipedia.org/wiki/Proximal_policy_optimization
- OpenAI. (n.d.). Proximal Policy Optimization (PPO) – Spinning Up in Deep RL. Retrieved March 16, 2026, from <https://spinningup.openai.com/en/latest/algorithms/ppo.html>
- Massachusetts Institute of Technology Professional Education. (n.d.). Advanced reinforcement learning. Retrieved March 4, 2026, from <https://professional.mit.edu/course-catalog/advanced-reinforcement-learning>
- YouTube. (n.d.). Advanced reinforcement learning video tutorial. Retrieved March 4, 2026, from <https://www.youtube.com/watch?v=9hbQieQh7-o>
- Roboflow. (n.d.). Clash Royale troop detection dataset. Retrieved February 13, 2026, from <https://universe.roboflow.com/nejc-zavodnik/clash-royale-troop-detection/dataset/13>
- Label Studio. (n.d.). Labeling tool for machine learning datasets. Retrieved September 10, 2025, from <https://labelstud.io/>
- GitHub. (n.d.). Clash Royale detection dataset. Retrieved December 3, 2025, from https://github.com/wty-vy/Clash-Royale-Detection-Dataset/blob/master/README_en.md

Questions?